Project Report, Iris and MNIST Classification

Jens Tjomsland (candidate nr. 10205)
Johannes Aasen Nesheim (candidate nr. 10180)

April 22, 2026

Contents

1 Introduction **2**

2 Theory **2**
2.1 The Linear Classifier . . . 2
2.2 The NN-classifier . . . 3
2.3 Evaluation . . . 4

3 Classifying Iris Species **4**
3.1 Design, Training and Generalization . . . 4
3.2 Features and Linear Separability . . . 4

4 Classifying Handwritten Numbers **4**
4.1 NN-Classifier . . . 4
4.2 Clustering with NN- and KNN-Classifiers . . . 4

5 Implementation **5**
5.1 Linear Classifier . . . 5
5.2 NN and KNN -Classifier . . . 5

6 Results **6**
6.1 Iris classification . . . 6
6.2 Handwritten Numbers Classification . . . 9

7 Conclusion **11**

8 References **12**

9 Appendix **12**
9.1 Code for Linear Classifier . . . 12
9.2 Code for NN-Classifier . . . 15
9.3 Code for NN with clustering . . . 17
9.4 Code for KNN with clustering . . . 18

Abstract

This report describes the implementation, theory and results behind using a linear classifier in order to classify different species of Iris flowers as well as a nearest neighbor based classifier for the classification of hand written numbers. We achieved at best a success-rate of 96.67% for the Iris flowers, and a success-rate of 96.91% for the handwritten numbers. The linear classifier performed equally well when only 1 feature was used, implying a high degree of linear separability for the Iris species. Introducing clustering for the nearest neighbor classifier reduced the performance slightly, but greatly reduced the run time of the classification.

1 Introduction

The project is part of the NTNU course Estimation, Detection and Classification (TTT4275). As state of the art classifiers are widely used within speech recognition and gene classification (Myrvoll, Werner, and Johnsen n.d., p. 73), understanding basic classifiers are important. The aim of this project is therefore to get an understanding of how different types of classifiers work, and how they can be used in practice. This is done by implementing a linear classifier for the classification of Iris species, and a nearest neighbor classifier (NN-classifier) for handwritten numbers from the MNIST database. The report describes the theory behind the linear classifier and the NN-classfier used in the project. The tasks are described in section 3 and 4, before the results are presented and analyzed in section 6. Some concluding remarks about the project are mentioned at the end 7.

2 Theory

The following section describes the theory behind the two classifiers used in this project. First, the theory behind the linear classifier is explained, and then the NN-classifier is described.

2.1 The Linear Classifier

As described in (Myrvoll, Werner, and Johnsen n.d., p. 69) the linear classifier uses a discriminant function

$$x \in \omega_j \iff g_j(x) = \max_i g_i(x) \tag{1}$$

in order to decide which class the instance x belongs to. The vector x contains values that define the features of the object being classified. In our case, the defining features that were used was the length and width of the petal and sepal of the Iris flower. For each class, the classifier has a linear function corresponding to that class. The function can be described as follows:

$$g_i(\mathbf{x}) = \mathbf{w}_i^T \mathbf{x} + w_{i0}, \quad i = 1, \ldots, C$$

where C corresponds to the number of classes. The linear classifier computes the value of the functions for each instance, and assigns the instance to the class corresponding to the function with the highest value. A critical component for the success of the classifier is assigning values to the weighting matrix $\mathbf{w}$. During training, an algebraic trick of defining a new weighting matrix W as follows

$$[\mathbf{W}\ \mathbf{w}_0] \to \mathbf{W}, \quad [\mathbf{x}^T\ 1]^T \to \mathbf{x}$$

allows us to optimize both the weighting matrix w and the bias term w_0 at the same time.

$$\Rightarrow \mathbf{g} = \mathbf{W}\mathbf{x}$$

By the use of gradient descent, the weighting matrix W can be updated by minimizing the Minimum Square Error (MSE) described below:

$$\text{MSE} = \frac{1}{2} \sum_{k=1}^{N} (\mathbf{g}_k - \mathbf{t}_k)^T (\mathbf{g}_k - \mathbf{t}_k)$$

The target vector t_k contains information about which class the instance x_k belongs to, by taking on a binary value 0,1 for each class. For instance, for an input x_k the output vector t_k may look like $\mathbf{t}_k = [1, 0, 0]^T$. Since the output from the linear function g_i can take on any continuous real value, we use a sigmoid function

$$g_{ik} = \text{sigmoid}(z_{ik}) = \frac{1}{1 + e^{-z_{ik}}}, \quad i = 1, \ldots, C$$

to map the output from g_i to a real value between 0 and 1, such that a comparison between g_k and t_k is possible. The weighting matrix W can now be updated by adjusting it in the opposite direction of the gradient. This process is repeated until a stopping criterion is met. In this project, training terminates after a fixed number of iterations, though other implementations may instead check for convergence. Each iteration of W is described by

$$W(m) = W(m-1) - \alpha \nabla_W \text{MSE}$$

In this equation, α denotes the step size taken in the gradient descent algorithm. The value of alpha was experimentally decided upon in this project, but other methods exist (Myrvoll, Werner, and Johnsen n.d., p. 80).
A linear classifier is sufficient when the classification problem is linearly separable (Myrvoll, Werner, and Johnsen n.d., p. 69). Linear separability is the degree to which it is possible to use a linear function to separate two sets of data(Myrvoll, Werner, and Johnsen n.d., p. 66). The classification of iris flowers is a problem that is almost linearly separable(*Project descriptions: Classification 1 2* n.d.), so a linear classifier is expected to perform well on this dataset.

2.2 The NN-classifier

The NN-classifier works by comparing the test sample to a set of templates given by the training set. By comparing the distance from the test sample to all training samples, or a representative set for each class (K-clustering), we can classify the new sample. Calculating the distance to all samples increases computational complexity, but results in accurate classifiers(Myrvoll, Werner, and Johnsen n.d., p. 70).
There are several possible decision rules, but in this project, euclidean distance was used (Myrvoll, Werner, and Johnsen n.d., p. 70). The decision rule is to assign the test sample to the class that has the least distance to the test sample. The euclidean distance is given by

$$d(\mathbf{x}, \mathbf{ref}_{ik}) = (\mathbf{x} - \boldsymbol{\mu}_{ik})^T (\mathbf{x} - \boldsymbol{\mu}_{ik})$$

When euclidean distance is used, μ is simply the same as the reference parameter ref_{ik}.

A more robust variant is the KNN classifier, where instead of finding only the single nearest reference, the $K > 1$ nearest references are found. The class label is then determined by a majority vote among these K neighbors. If several classes receive the same number of votes, the closest class among them is chosen(Myrvoll, Werner, and Johnsen n.d., p. 70). This makes the classifier more robust against outlier templates that may be close to the test sample.
Clustering reduces the number of templates by grouping similar training samples together and representing each group by its mean vector. The clustering algorithm partitions the training vectors into M clusters, where each cluster center $\boldsymbol{\mu}_{ik}$ is the mean of all training samples assigned to that cluster(Myrvoll, Werner, and Johnsen n.d., p. 81). These cluster centers serve as the new reference templates $\mathbf{ref}_{ik} = \boldsymbol{\mu}_{ik}$. It achieves this by initializing one cluster for each class. Then the algorithm computes the distance to the cluster and assigns the training samples to that cluster. Then an iterative process starts where where the clusters are split, distances and cluster centers are recomputed, and training samples are reassigned. Most commonly, and in this project, the clusters with the most training samples are split. This repeats until the cluster centers converge. This means fewer distance calculations are needed, while each class is still well represented by its templates.

2.3 Evaluation

The two metrics used to evaluate the performance of the classifiers were the confusion matrix and error rate. The confusion matrix

$$\mathbf{C} = \begin{pmatrix} c_{11} & c_{12} & \cdots & c_{1C} \\ c_{21} & c_{22} & \cdots & c_{2C} \\ \vdots & \vdots & \ddots & \vdots \\ c_{C1} & c_{C2} & \cdots & c_{CC} \end{pmatrix}$$

shows correctly classified samples on the diagonal and misclassified samples on the off-diagonal elements. A non-zero element at c_{ij} indicates that the classifier guessed class j while the correct label is class i. The error rate is defined as

$$\text{Error rate} = \frac{\text{Number of misclassified samples}}{\text{Total number of samples}} = \frac{\sum_{i \neq j} c_{ij}}{\sum_{i=1}^{C} \sum_{j=1}^{C} c_{ij}}$$

3 Classifying Iris Species

3.1 Design, Training and Generalization

The tasks of this part of the project consisted of splitting a data set of the length and width of the three species of Iris flower into a training and test set, consisting of 30 and 20 samples respectively. Then a linear classifier had to be made and evaluated using error rate and confusion matrices for both the training and test sets. Lastly, the training and test samples were switched such that the first 20 samples were used for testing, and the last 30 for training (*Project descriptions: Classification 1 2* n.d., p. 1). The results are shown and discussed in 6.

3.2 Features and Linear Separability

The project also included tasks examining the discriminative properties of the four Iris features. Histograms were produced for each feature across all three classes to visually assess class overlap. The feature exhibiting the greatest overlap between classes was identified and removed, and the linear classifier was retrained and evaluated using the remaining three features. This procedure was repeated successively, until classifiers using only two and one features had also been trained and evaluated.

4 Classifying Handwritten Numbers

This section describes the tasks involved in the second part of the project which entailed the classification of handwritten numbers. Each image consists of 28×28 pixels in 8-bit grayscale, and the dataset contains 60 000 training examples and 10 000 test examples written by different individuals.

4.1 NN-Classifier

Using the MNIST dataset of handwritten numbers, we were to design a NN-classifier using Euclidean distance, where we used the entire training set as templates. To manage computational load, the test set was processed in chunks of 1 000 images at a time when computing distances. After plotting both misclassified and correctly classified numbers, we analyzed the handwritten numbers using our own interpretation.

4.2 Clustering with NN- and KNN-Classifiers

In the second part of the classification of handwritten numbers, k-means clustering was applied independently to the 6 000 training vectors of each class in order to reduce the number of templates from 6 000 per class to $M = 64$ cluster centers per class, yielding a total of 640 templates. The NN classifier was then retrained using these reduced templates. Building on the clustered template set, a K-nearest neighbor (KNN) classifier with $K = 7$ was implemented.

5 Implementation

The implementation of the classifiers and python program to solve the tasks is given by the flow charts below. The complete python scripts are shown in the Appendix.

5.1 Linear Classifier

The python machine learning library *pyTorch* was used to implement the linear classifier in accordance with the flow chart in 1(PyTorch Contributors 2025).

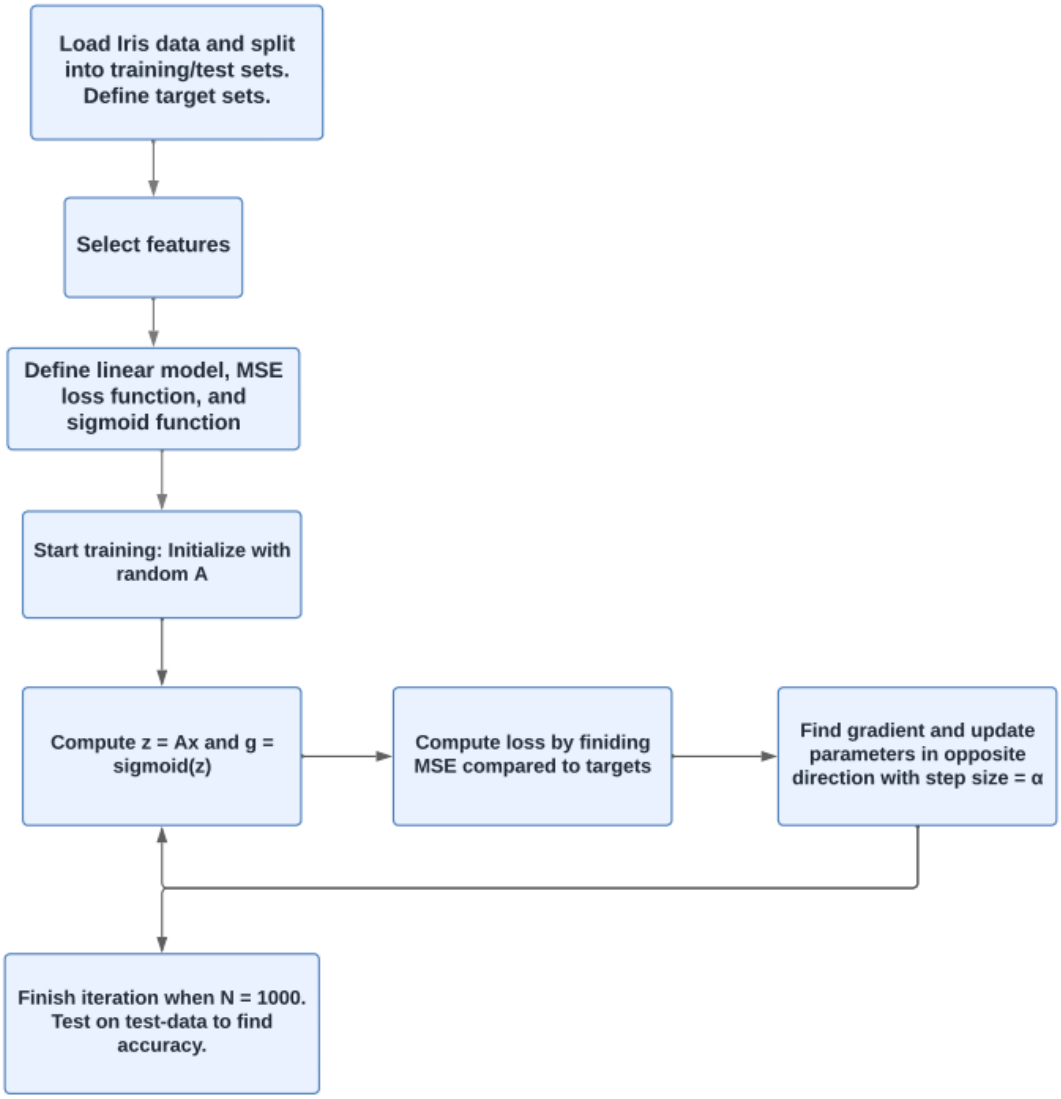

Figure 1: Program flow

5.2 NN and KNN -Classifier

The flow chart below describes the implementation of the three python scripts implemented to solve the tasks. The *Kmeans* function and *confusion matrix* function from the Scikit-learn library were used for clustering and plotting (scikit-learn Contributors 2025).

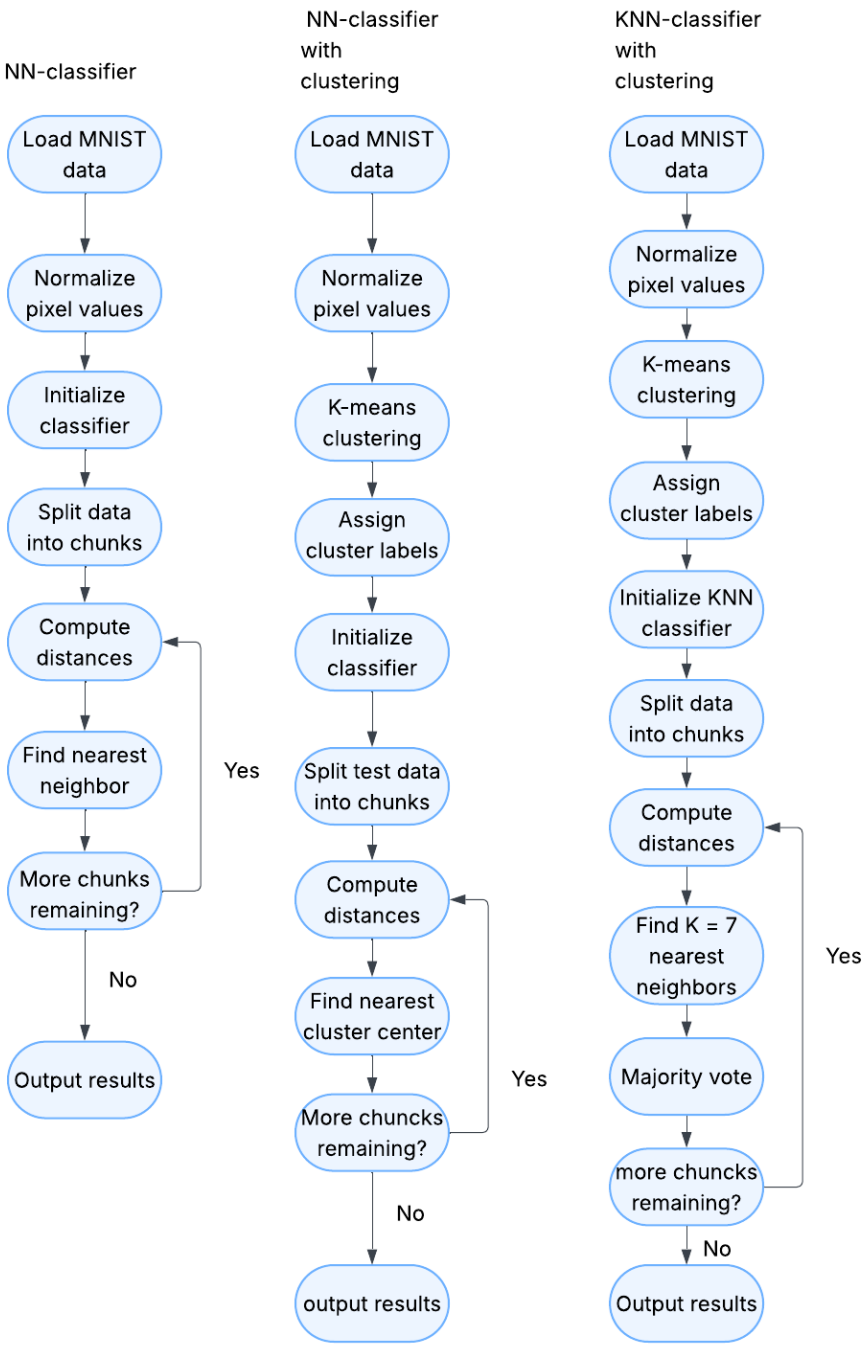

Figure 2: Program flow

6 Results

In the following section the results of running our classifiers are shown and discussed.

6.1 Iris classification

To find a suitable step size α, a cut and try approach was used. Different values of α were tested iteratively until a value of $\alpha = 0.2$ was found, at which point the gradient descent algorithm converged.

In the following tables Iris Setosa, Iris Versicolor and Iris Virginica are abbreviated with ST, VC and VG.

	ST	**VC**	**VG**
ST	30	0	0
VC	0	29	1
VG	0	1	29

Training Set (Error rate: 2.22%)

	ST	**VC**	**VG**
ST	19	1	0
VC	0	19	1
VG	0	1	19

Test Set (Error rate: 5.00%)

Table 1: First 30 samples for training, last 20 for testing

	ST	**VC**	**VG**
ST	30	0	0
VC	0	28	2
VG	0	2	28

Training Set (Error rate: 4.44%)

	ST	**VC**	**VG**
ST	20	0	0
VC	0	20	0
VG	0	2	18

Test Set (Error rate: 3.33%)

Table 2: Last 30 samples for training, first 20 for testing

Both training configurations have low error rates, indicating that the linear classifier generalizes well. In Case 1, slightly better training performance is observed, while Case 2 shows a small increase in training error rate , but lower test error rate. A possible explanation for this is that the data samples towards the end of the dataset are less linearly separable, and thus harder to classify. The confusion matrices indicate that most misclassifications occur between Class 2 and Class 3, suggesting some overlap between these classes. Overall, the classifier demonstrates stable performance regardless of how the dataset is split, with only minor variation.

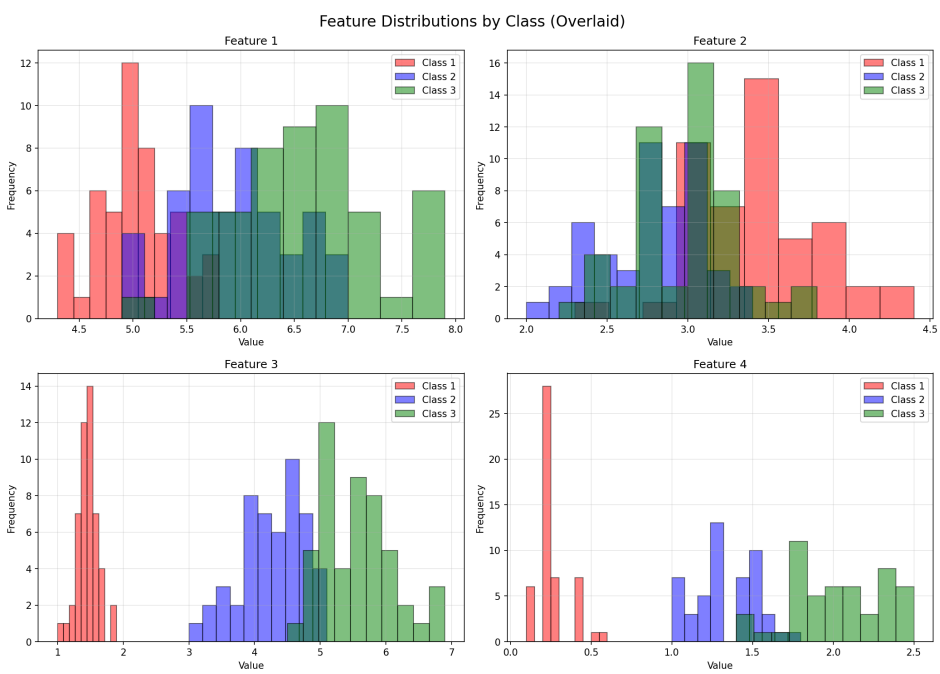

Figure 3: Histogram of the overlap between the features of the Iris species.

To find the most discriminative features, we used histograms to visualize the separation and overlap between the features. Based on the histograms, feature 1 and 2 showed the most overlap between

classes, while features 3 and 4 exhibited the least overlap and was therefore considered the most discriminative. We tried retraining the classifier with fewer classes, starting by removing the classes that showed the most overlap.

	ST	**VC**	**VG**
ST	30	0	0
VC	0	28	2
VG	0	1	29

Training Set (Error rate: 3.33%)

	ST	**VC**	**VG**
ST	20	0	0
VC	0	19	1
VG	0	1	19

Test Set (Error rate: 3.33%)

Table 3: Removed Feature 2

	ST	**VC**	**VG**
ST	30	0	0
VC	0	28	2
VG	0	1	29

Training Set (Error rate: 3.33%)

	ST	**VC**	**VG**
ST	20	0	0
VC	0	20	0
VG	0	2	18

Test Set (Error rate: 3.33%)

Table 4: Removed Features 1 and 2

	ST	**VC**	**VG**
ST	30	0	0
VC	0	27	3
VG	0	2	28

Training Set (Error rate: 5.56%)

	ST	**VC**	**VG**
ST	20	0	0
VC	0	19	1
VG	0	1	19

Test Set (Error rate: 3.33%)

Table 5: Removed Features 1, 2 and 4

Although the overlap between the species for feature 3 is low, the classifier surprisingly performed equally well when removing the other features from the dataset. This Indicates that feature 3 has a high degree of linear separability between the 3 species, such that the classifier can rely almost exclusively on this feature when deciding the class.

6.2 Handwritten Numbers Classification

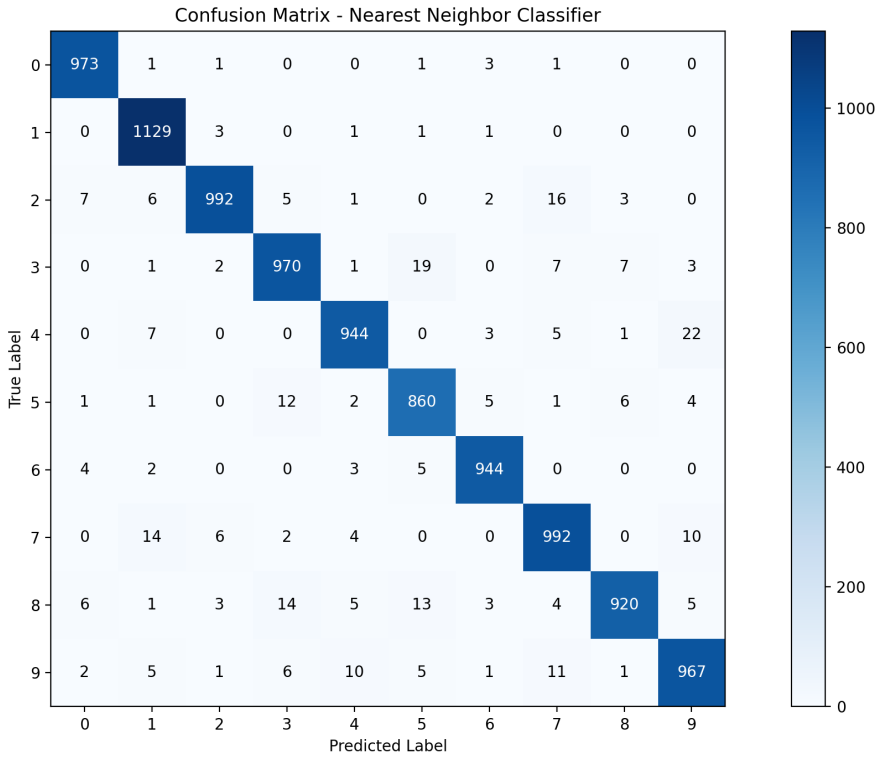

Figure 4: Confusion matrix for NN-classifier without clustering.

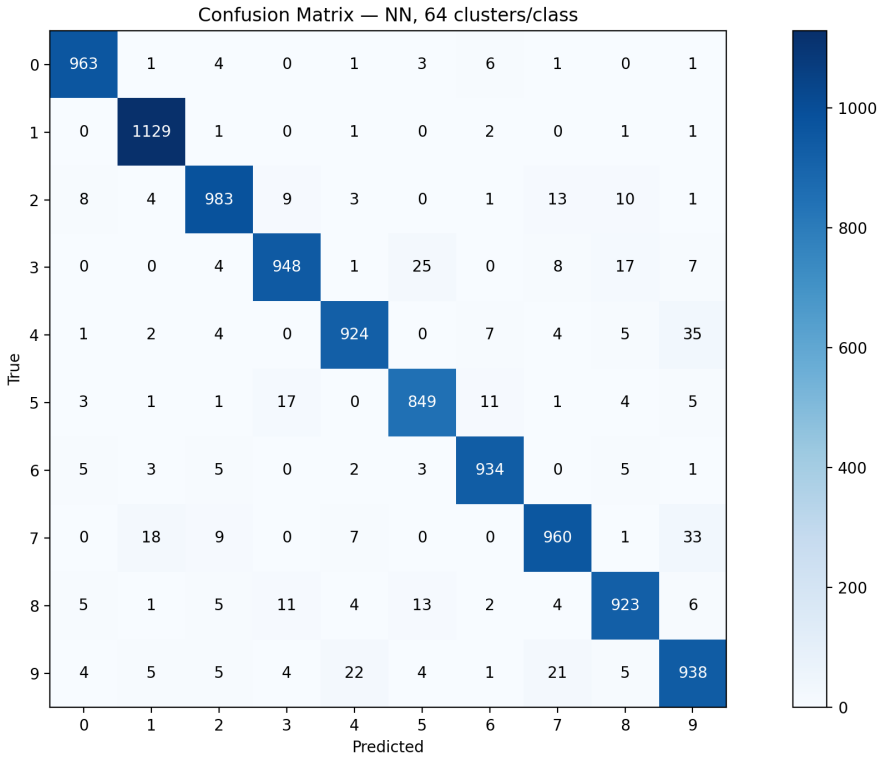

Figure 5: Confusion matrix for NN-classifier with the use of clustering.

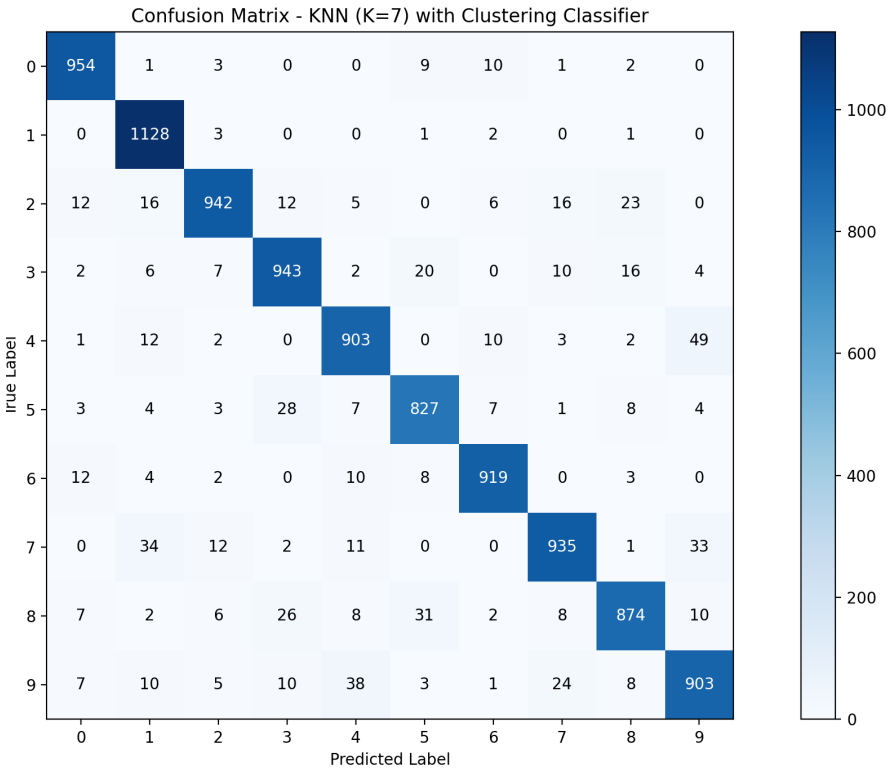

Figure 6: Confusion matrix for KNN-classifier with the use of clustering.

Figure 7: Correctly classified numbers using NN-classifier.

Figure 8: Misclassified numbers using NN-classifier.

Figure 9: Misclassified numbers using NN-classifier.

Overall, the different types of template based classifiers exhibited good performance. The NN-classifier without clustering performs worse than our own human classification, but the errors were understandable. Especially test 321 and test 358 in figure 8 are non-trivial test samples that we think humans also could make errors classifying. A potential reason for the misclassifications is that parts of the samples are extremely similar to other samples of a different class. Since the classifier only looks at the total distance, it's not able to distinguish between what features of the samples are important. For example, the classifier does not know what to look for when distinguishing an 8 from a 9 (i.e. 8 has two circles, 9 does not). Therefore, when a sample is really similar, but differs in some essential parts, it fails to distinguish them, and errors can occur. This is a probable explanation for the mislabel of test 195 in 9 where the bottom half of the numbers are almost identical, but differ in the top half.

Classifier	**Error rate (%)**	**Clustering Time (s)**	**Prediction Time (s)**
NN	3.09	—	9.30
NN with clustering	4.49	261.12	0.21
KNN with clustering	6.72	242.95	0.40

Table 6: Performance comparison of the three classifiers on the MNIST test set.

Introducing clustering made the error rate marginally worse. By reducing the amount of templates, you discard some of the information in your data. Samples that closely align with the test-sample could be grouped into a cluster further away than the individual samples would have been, which can cause misclassifications.

The clustering algorithm is time-intensive, which means the classifiers with clustering needed much more time for training. However, the prediction time was much faster. Therefore, using clustering in time dependent applications may be beneficial. For applications where speed is less important, choosing a KNN - classifier without the use of clustering seems like a robust,logical choice.

Interestingly, the KNN classifier with the use of clustering performed worse than without a K-majority vote. We believe the reason for this is that the clustering algorithm removes the outliers that a KNN classifier usually would "protect" against. Since the clusters that are used as templates are computed by using the mean, outliers are already "removed" from the reference set. Therefore, a K-majority voting system may incorrectly include clusters from a different class when the test sample lies near the boundary between cluster groups.

7 Conclusion

This project provided hands-on experience with the linear classifier and template-based nearest neighborhood classifiers. For the Iris classification task, the linear classifier demonstrated strong and stable performance across both training and test splits, achieving a best test error rate of 3.33%. The feature analysis and subsequent testing revealed that retraining the classifier with only feature 3 preserved the same test error rate as using all four features. This demonstrated to us that more features do not

necessarily improve classifier performance, and that identifying the most discriminative features can be an effective way to reduce complexity while maintaining performance.

For the handwritten digit classification task, the standard NN classifier achieved the lowest error rate of 3.09%. The classifier with clustering only performed marginally worse, while cutting the classification time from 9.30 to 0.21 seconds. We learned that training a classifier with clustering is costly with regard to time, but will make faster predictions, and is therefore better suited for implementations with time-constraints.

Overall, the project reinforced that the choice of classifier and its configuration should be guided by the specific requirements of the application. When prediction speed matters, clustering offers a compelling trade-off. When maximum accuracy is the priority, using the full training set as templates is the more reliable choice. In addition, we learned that trying to implement too much robustness with regard to outliers can negatively impact performance, as we witnessed when using a KNN-classifier with clustering.

8 References

Myrvoll, Tor A., Stefan Werner, and Magne H. Johnsen (n.d.). *Estimation, Detection and Classification Theory.*

Project descriptions: Classification 1 2 (n.d.).

PyTorch Contributors (2025). *torch.nn.Linear — PyTorch 2.10 Documentation.* https://docs.pytorch.org/docs/stable/generated/torch.nn.Linear.html. Accessed: 2026-04-21.

scikit-learn Contributors (2025). *API Reference — scikit-learn 1.6.1 documentation.* Accessed: 2026-04-21. URL: https://scikit-learn.org/stable/api/index.html.

9 Appendix

9.1 Code for Linear Classifier

```
import torch
import numpy as np

# Load class data files
class_1 = np.loadtxt('class_1', delimiter=',')
class_2 = np.loadtxt('class_2', delimiter=',')
class_3 = np.loadtxt('class_3', delimiter=',')

# Extract first 30 samples from each class for training
class_1_training = class_1[:30]
class_2_training = class_2[:30]
class_3_training = class_3[:30]

# Extract last 20 samples from each class for testing
class_1_test = class_1[30:]
class_2_test = class_2[30:]
class_3_test = class_3[30:]

# Combine all training and test features
X_train = np.vstack([class_1_training, class_2_training, class_3_training])
X_test = np.vstack([class_1_test, class_2_test, class_3_test])

# Select wanted features
X_train = X_train[:, [0, 1, 2, 3]]
X_test = X_test[:, [0, 1, 2, 3]]
```

```python
# Hardcode labels
t_train = np.vstack([
    np.concatenate([[1]*30, [0]*30, [0]*30]),
    np.concatenate([[0]*30, [1]*30, [0]*30]),
    np.concatenate([[0]*30, [0]*30, [1]*30]),
])
t_test = np.vstack([
    np.concatenate([[1]*20, [0]*20, [0]*20]),
    np.concatenate([[0]*20, [1]*20, [0]*20]),
    np.concatenate([[0]*20, [0]*20, [1]*20]),
])

# Convert the data to PyTorch tensors
X_train = torch.tensor(X_train).float()
X_test = torch.tensor(X_test).float()
t_train = torch.tensor(t_train).float()
t_test = torch.tensor(t_test).float()

# Normalize the features
mean = X_train.mean(dim=0)
std = X_train.std(dim=0)
X_train = (X_train - mean) / std
X_test = (X_test - mean) / std

# Define the model
model = torch.nn.Linear(in_features = 4, out_features =3, bias = True)

def MSE(g, t):
    return 0.5 * torch.sum((g - t) ** 2)

# Train the model with manual gradient descent
sigmoid = torch.nn.Sigmoid()
optimizer = torch.optim.SGD(model.parameters(), lr=0.2)

N = 1000
for i in range(N):
    # Forward pass
    z = model(X_train).t()
    g = sigmoid(z)
    loss = MSE(g, t_train)
    optimizer.zero_grad()   # Clear old gradients
    loss.backward()         # Compute gradients for all params
    optimizer.step()        # Update weights using those gradients

    # Print the loss every 100 iterations
    if (i+1) % 100 == 0:
        print(f'Iteration [{i+1}/{N}], Loss: {loss.item():.4f}')


# Create confusion matrices using final parameters
print("\n" + "CONFUSION MATRIX - TRAINING SET")
# Get predictions on training set
z_train = model(X_train).t()
g_train = sigmoid(z_train)

# Convert predictions to class labels (argmax)
predictions_train = torch.argmax(g_train, dim=0).numpy() + 1
actual_train = torch.argmax(t_train, dim=0).numpy() + 1
```

```
# Create confusion matrix for training
confusion_matrix_train = np.zeros((3, 3), dtype=int)
for i in range(len(predictions_train)):
    confusion_matrix_train[actual_train[i] - 1, predictions_train[i] - 1] += 1
print("        Class 1  Class 2  Class 3")
for i in range(3):
    print(f"Class {i+1}  {confusion_matrix_train[i, 0]:5d}   {confusion_matrix_train[i, 1]:5d}   {con

# Calculate training accuracy
accuracy_train = np.trace(confusion_matrix_train) / np.sum(confusion_matrix_train)
print(f"\nTraining Accuracy: {accuracy_train:.4f} ({100*accuracy_train:.2f}%)")

print("\n" + "CONFUSION MATRIX - TEST SET")

# Get predictions on test set
z_test = model(X_test).t()
g_test = sigmoid(z_test)

# Convert predictions to class labels (argmax)
predictions = torch.argmax(g_test, dim=0).numpy() + 1  # +1 to match class numbers 1, 2, 3
actual = torch.argmax(t_test, dim=0).numpy() + 1

# Create confusion matrix
confusion_matrix = np.zeros((3, 3), dtype=int)
for i in range(len(predictions)):
    confusion_matrix[actual[i] - 1, predictions[i] - 1] += 1

print("        Class 1  Class 2  Class 3")
for i in range(3):
    print(f"Class {i+1}  {confusion_matrix[i, 0]:5d}   {confusion_matrix[i, 1]:5d}   {confusion_matri

# Calculate accuracy
accuracy = np.trace(confusion_matrix) / np.sum(confusion_matrix)
print(f"\nTest Accuracy: {accuracy:.4f} ({100*accuracy:.2f}%)")

# Create histograms for each feature and class
print("FEATURE HISTOGRAMS")

feature_names = ['Feature 1', 'Feature 2', 'Feature 3', 'Feature 4']
class_names = ['Class 1', 'Class 2', 'Class 3']
class_data = [class_1, class_2, class_3]
colors = ['red', 'blue', 'green']

# Create a figure with subplots (4 features x 3 classes)
fig, axes = plt.subplots(4, 3, figsize=(15, 12))
fig.suptitle('Feature Distributions by Class', fontsize=16)

for feature_idx in range(4):
    for class_idx in range(3):
        ax = axes[feature_idx, class_idx]
        ax.hist(class_data[class_idx][:, feature_idx], bins=10, color=colors[class_idx], alpha=0.7, e
        ax.set_title(f'{feature_names[feature_idx]} - {class_names[class_idx]}')
        ax.set_xlabel('Value')
        ax.set_ylabel('Frequency')
        ax.grid(alpha=0.3)

plt.tight_layout()
```

```
plt.savefig('feature_histograms2.png', dpi=150, bbox_inches='tight')
plt.show()

print("\nHistogram plot saved as 'feature_histograms.png'")
```

9.2 Code for NN-Classifier

```
import numpy as np
import scipy.io as sio
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
import time


class NNClassifier:
    def __init__(self, train_data, train_labels):
        self.templates = train_data.astype(np.float32)
        self.labels = train_labels
        # precompute norms once to speed up distance calculations later
        self.norms = np.sum(self.templates ** 2, axis=1, keepdims=True)

    def _sq_distances(self, chunk):
        # ||a-b||^2 = ||a||^2 + ||b||^2 - 2(a.b), avoids computing sqrt
        chunk_norms = np.sum(chunk ** 2, axis=1, keepdims=True)
        dot = np.dot(chunk, self.templates.T)
        return np.maximum(chunk_norms + self.norms.T - 2 * dot, 0)

    def predict(self, test_data, chunk_size=1000):
        n = test_data.shape[0]
        preds = np.zeros(n, dtype=int)
        nn_idx = np.zeros(n, dtype=int)
        nn_dist = np.zeros(n)

        for i in range(0, n, chunk_size):
            chunk = test_data[i:i + chunk_size]
            dists = self._sq_distances(chunk)
            nearest = np.argmin(dists, axis=1)
            preds[i:i + len(chunk)] = self.labels[nearest]
            nn_idx[i:i + len(chunk)] = nearest
            nn_dist[i:i + len(chunk)] = dists[np.arange(len(chunk)), nearest]

        return preds, nn_idx, nn_dist


def show_confusion_matrix(cm):
    plt.figure(figsize=(10, 8))
    plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
    plt.title('Confusion Matrix | Nearest Neighbor Classifier')
    plt.colorbar()
    plt.xlabel('Predicted')
    plt.ylabel('True')

    thresh = cm.max() / 2
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(j, i, cm[i, j], ha='center', va='center',
                     color='white' if cm[i, j] > thresh else 'black')
```

```
    plt.xticks(range(10))
    plt.yticks(range(10))
    plt.tight_layout()
    plt.show()


def show_samples(test_data, test_labels, preds, nn_idx, nn_dist, train_data, n=9, correct=True):
    mask = test_labels == preds if correct else test_labels != preds
    indices = np.where(mask)[0][:n]
    label_color = 'green' if correct else 'red'

    n_cols = min(3, len(indices))
    n_rows = int(np.ceil(len(indices) / n_cols))

    fig, axes = plt.subplots(n_rows, n_cols * 2, figsize=(12, 4 * n_rows))
    if n_rows == 1:
        axes = axes.reshape(1, -1)

    for i, idx in enumerate(indices):
        row, col = i // n_cols, (i % n_cols) * 2

        axes[row, col].imshow(test_data[idx].reshape(28, 28), cmap='gray')
        axes[row, col].set_title(
            f'Test {idx}\nTrue: {test_labels[idx]}, Pred: {preds[idx]}',
            color=label_color, fontweight='bold'
        )
        axes[row, col].axis('off')

        axes[row, col + 1].imshow(train_data[nn_idx[idx]].reshape(28, 28), cmap='gray')
        axes[row, col + 1].set_title(
            f'Neighbor {nn_idx[idx]}\nLabel: {train_labels[nn_idx[idx]]}\nDist: {nn_dist[idx]:.2f}',
            fontweight='bold'
        )
        axes[row, col + 1].axis('off')

    # hide unused subplots
    for j in range(len(indices), n_rows * n_cols):
        r, c = j // n_cols, (j % n_cols) * 2
        if r < n_rows:
            axes[r, c].axis('off')
            axes[r, c + 1].axis('off')

    plt.tight_layout()
    plt.show()


data = sio.loadmat('data_all.mat')

train_data   = data['trainv'].astype(np.float32) / 255.0
train_labels = data['trainlab'].flatten().astype(int)
test_data    = data['testv'].astype(np.float32) / 255.0
test_labels  = data['testlab'].flatten().astype(int)

clf = NNClassifier(train_data, train_labels)

print("\nrunning NN on test set...")
t0 = time.time()
preds, nn_idx, nn_dist = clf.predict(test_data)
```

```
print(f"done in {time.time() - t0:.1f}s")

cm = confusion_matrix(test_labels, preds)
acc = np.diag(cm).sum() / cm.sum()
print(f"\naccuracy:   {acc:.4f}")
print(f"error rate: {1 - acc:.4f}")
print("\nconfusion matrix:")
print(cm)

show_confusion_matrix(cm)
show_samples(test_data, test_labels, preds, nn_idx, nn_dist, train_data, correct=False)
show_samples(test_data, test_labels, preds, nn_idx, nn_dist, train_data, correct=True)
```

9.3 Code for NN with clustering

```
    from sklearn.cluster import KMeans
from sklearn.metrics import confusion_matrix
import numpy as np
import scipy.io as sio
import matplotlib.pyplot as plt
import time

M = 64


class NNClassifier:
    def __init__(self, templates, labels):
        self.templates = templates.astype(np.float32)
        self.labels = labels
        # precompute norms once to speed up distance calculations later
        self.norms = np.sum(self.templates ** 2, axis=1, keepdims=True)

    def _sq_distances(self, chunk):
        # ||a-b||^2 = ||a||^2 + ||b||^2 - 2(a.b), avoids computing sqrt
        chunk_norms = np.sum(chunk ** 2, axis=1, keepdims=True)
        dot = np.dot(chunk, self.templates.T)
        return np.maximum(chunk_norms + self.norms.T - 2 * dot, 0)

    def predict(self, test_data, chunk_size=5000):
        n = test_data.shape[0]
        preds = np.zeros(n, dtype=int)

        for i in range(0, n, chunk_size):
            chunk = test_data[i:i + chunk_size]
            dists = self._sq_distances(chunk)
            preds[i:i + len(chunk)] = self.labels[np.argmin(dists, axis=1)]

        return preds


def build_templates(train_data, train_labels, m=M):
    classes = np.unique(train_labels)
    centers, center_labels = [], []

    for cls in classes:
        km = KMeans(n_clusters=m, random_state=42, n_init=3, max_iter=300)
        km.fit(train_data[train_labels == cls])
        centers.append(km.cluster_centers_)
```

```
        center_labels.extend([cls] * m)

    template_center = np.vstack(centers).astype(np.float32)
    template_label = np.array(center_labels, dtype=int)
    return template_center, template_label


def show_confusion_matrix(cm):
    plt.figure(figsize=(10, 8))
    plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
    plt.title(f'Confusion Matrix | NN, {M} clusters/class')
    plt.colorbar()
    plt.xlabel('Predicted')
    plt.ylabel('True')

    thresh = cm.max() / 2
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(j, i, cm[i, j], ha='center', va='center',
                     color='white' if cm[i, j] > thresh else 'black')

    plt.xticks(range(10))
    plt.yticks(range(10))
    plt.tight_layout()
    plt.show()


data = sio.loadmat('data_all.mat')

train_data   = data['trainv'].astype(np.float32) / 255.0
train_labels = data['trainlab'].flatten().astype(int)
test_data    = data['testv'].astype(np.float32) / 255.0
test_labels  = data['testlab'].flatten().astype(int)
clustering_time = time.time()
print(f"\nbuilding {M} templates per class via k-means...")
templates, t_labels = build_templates(train_data, train_labels)
print(f"Clustering done in {time.time() - clustering_time:.2f}s")
print(f"total templates: {templates.shape[0]}")

clf = NNClassifier(templates, t_labels)

print("\nrunning NN on test set...")
prediction_time = time.time()
preds = clf.predict(test_data)
print(f"Prediction done in {time.time() - prediction_time:.2f}s")

cm = confusion_matrix(test_labels, preds)
acc = np.diag(cm).sum() / cm.sum()
print(f"\naccuracy:   {acc:.4f}")

show_confusion_matrix(cm)
```

9.4 Code for KNN with clustering

```
    from sklearn.cluster import KMeans
from sklearn.metrics import confusion_matrix
import numpy as np
import scipy.io as sio
```

```python
import matplotlib.pyplot as plt
import time

K = 7
M = 64


class KNNClassifier:
    def __init__(self, templates, labels, k=K):
        self.templates = templates.astype(np.float32)
        self.labels = labels
        self.k = k
        # precompute norms once to speed up distance calculations later
        self.norms = np.sum(self.templates ** 2, axis=1, keepdims=True)

    def _sq_distances(self, chunk):
        # ||a-b||^2 = ||a||^2 + ||b||^2 - 2(a.b), avoids computing sqrt
        chunk_norms = np.sum(chunk ** 2, axis=1, keepdims=True)
        dot = np.dot(chunk, self.templates.T)
        return np.maximum(chunk_norms + self.norms.T - 2 * dot, 0)

    def predict(self, test_data, chunk_size=5000):
        n = test_data.shape[0]
        preds = np.zeros(n, dtype=int)

        for i in range(0, n, chunk_size):
            chunk = test_data[i:i + chunk_size]
            dists = self._sq_distances(chunk)

            # grab k nearest for each sample, majority vote decides label
            knn_idx = np.argpartition(dists, self.k, axis=1)[:, :self.k]
            knn_labels = self.labels[knn_idx]
            for j in range(len(chunk)):
                preds[i + j] = np.bincount(knn_labels[j]).argmax()

        return preds


def build_templates(train_data, train_labels, m=M):
    classes = np.unique(train_labels)
    centers, center_labels = [], []

    for cls in classes:
        km = KMeans(n_clusters=m, random_state=42, n_init=3, max_iter=300)
        km.fit(train_data[train_labels == cls])
        centers.append(km.cluster_centers_)
        center_labels.extend([cls] * m)

    template_center = np.vstack(centers).astype(np.float32)
    template_label =np.array(center_labels, dtype=int)
    return template_center, template_label


def show_confusion_matrix(cm):
    plt.figure(figsize=(10, 8))
    plt.imshow(cm, interpolation='nearest', cmap=plt.cm.Blues)
    plt.title(f'Confusion Matrix | KNN (K={K}), {M} clusters/class')
    plt.colorbar()
```

```
    plt.xlabel('Predicted')
    plt.ylabel('True')

    thresh = cm.max() / 2
    for i in range(cm.shape[0]):
        for j in range(cm.shape[1]):
            plt.text(j, i, cm[i, j], ha='center', va='center',
                     color='white' if cm[i, j] > thresh else 'black')

    plt.xticks(range(10))
    plt.yticks(range(10))
    plt.tight_layout()
    plt.show()


data = sio.loadmat('data_all.mat')

train_data   = data['trainv'].astype(np.float32) / 255.0
train_labels = data['trainlab'].flatten().astype(int)
test_data    = data['testv'].astype(np.float32) / 255.0
test_labels  = data['testlab'].flatten().astype(int)

print(f"\nbuilding {M} templates per class via k-means...")
clustering_time = time.time()
templates, t_labels = build_templates(train_data, train_labels)
print(f"Clustering done in {time.time() - clustering_time:.2f}s")
print(f"total templates: {templates.shape[0]}")


clf = KNNClassifier(templates, t_labels, k=K)

print("\nrunning KNN on test set...")
prediction_time = time.time()
preds = clf.predict(test_data)
print(f"Prediction done in {time.time() - prediction_time:.2f}s")


cm = confusion_matrix(test_labels, preds)
acc = np.diag(cm).sum() / cm.sum()
print(f"\naccuracy:   {acc:.4f}")

show_confusion_matrix(cm)
```